

Casa Intelligente IoT con MQTT su ESP32

Relazione tecnica di progetto

Giacomo Masiero — ITT Marconi Rovereto

Maggio 2026

1. Introduzione

1.1 Obiettivo del progetto

Realizzare un sistema **IoT distribuito** per una “casa intelligente” basato sul protocollo **MQTT**. Il sistema deve simulare un ambiente domestico in cui più dispositivi indipendenti collaborano scambiandosi dati tramite un **broker** centrale, secondo il modello **publish/subscribe**.

Il sistema è composto da quattro nodi autonomi:

1. **Sensore di temperatura** — pubblica letture periodiche;
2. **Sensore di luminosità** — pubblica letture periodiche;
3. **Termostato** — sottoscrive il topic della temperatura, controlla soglie e genera allarmi;
4. **Monitor centralizzato** — sottoscrive con wildcard tutti gli eventi della casa e li registra.

A questi si aggiunge il **broker MQTT** (Mosquitto), che funge da centralino dei messaggi.

1.2 Tecnologie utilizzate

Componente	Tecnologia	Motivazione
Microcontrollore	ESP32 DevKit v1	WiFi integrato, dual-core, libreria Arduino matura
Linguaggio	C/C++ (framework Arduino)	Standard per ESP32, ampia disponibilità di librerie
Protocollo applicativo	MQTT v3.1.1	Pensato per IoT: leggero, asincrono, basato su TCP
Libreria MQTT	PubSubClient di Nick O’Leary	De-facto standard su Arduino/ESP
Broker	Mosquitto su Raspberry Pi	Open source, leggero, configurazione minima
Trasporto	TCP/IP su WiFi (porta 1883)	Standard MQTT non cifrato per LAN

1.3 Cosa rende “robusto” il sistema

Le richieste della consegna parlano di una “robusta gestione degli errori”. Nel progetto questo si traduce in:

- riconnessione automatica WiFi e MQTT con backoff,
- Last Will & Testament per notificare disconnessioni anomale,
- validazione di tutti i payload ricevuti,
- buffer a dimensione fissa (niente allocazioni dinamiche in hot path),
- QoS 1 sulle sottoscrizioni critiche,
- keep-alive con timeout per rilevare connessioni “fantasma”.

I dettagli sono spiegati nelle sezioni successive.

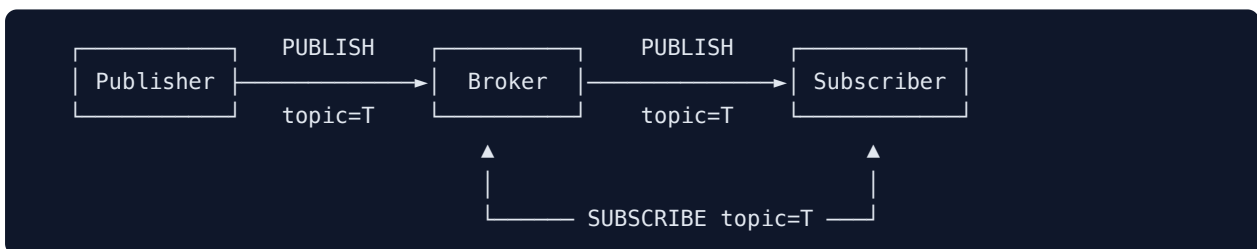
2. Il protocollo MQTT

2.1 Cos'è MQTT e perché si usa nell'IoT

MQTT (Message Queuing Telemetry Transport) è un protocollo applicativo di messaggistica progettato originariamente per applicazioni a banda limitata e dispositivi con poche risorse. Le sue caratteristiche principali:

- **Pattern publish/subscribe:** chi produce dati (publisher) e chi li consuma (subscriber) **non si conoscono**. Comunicano attraverso un intermediario chiamato **broker**.
- **Topic:** i messaggi sono organizzati in canali gerarchici tipo file system (`casa/salotto/temperatura`).
- **Asincrono e basato su TCP:** la connessione resta aperta e i messaggi viaggiano in entrambe le direzioni quando servono.
- **Header minimale:** l'overhead del protocollo è di soli 2 byte per i messaggi più piccoli, rendendolo adatto a microcontrollori e reti lente.

2.2 Il modello publish/subscribe



Differenze rispetto a un classico client/server HTTP:

	HTTP request/response	MQTT publish/subscribe
Iniziativa	Sempre del client	Chiunque, in qualunque momento
Connessione	Aperta e chiusa per ogni richiesta	Persistente
Accoppiamento	Client conosce il server	Publisher e subscriber non si conoscono
Direzione	Unidirezionale per richiesta	Full duplex sulla stessa connessione

Questo disaccoppiamento è cruciale nell'IoT: se aggiungo un nuovo sensore o una nuova dashboard, non devo riconfigurare gli altri nodi. Si abbonano semplicemente al topic giusto.

2.3 Anatomia di un topic

Un topic è una stringa con livelli separati da `/`. Esempi nel nostro progetto:

```
casa/salotto/temperatura
casa/salotto/luce
casa/salotto/termostato/allarme
casa/salotto/temperatura/status
```

Regole:

- non iniziano con `/`;
- sono case-sensitive;
- non hanno un significato intrinseco per il broker: sono solo etichette, ma la convenzione di scrivere `<luogo>/<dispositivo>/<grandezza>` aiuta moltissimo a usare le wildcard.

Wildcard nelle sottoscrizioni

Sono due caratteri speciali ammessi solo lato subscribe:

- `+` sostituisce esattamente UN livello.
 - `casa/+/temperatura` riceve `casa/salotto/temperatura`, `casa/cucina/temperatura`, ecc.
- `#` sostituisce ZERO o più livelli e deve stare alla fine.
 - `casa/#` riceve tutto ciò che inizia per `casa/`.

Nel nostro progetto il **monitor** usa proprio `casa/#` per registrare ogni evento.

2.4 QoS (Quality of Service)

MQTT prevede tre livelli di consegna garantita. La scelta è un trade-off fra affidabilità e overhead di rete.

QoS	Nome	Comportamento	Quando si usa
0	At most once	“Fire and forget”: il messaggio parte una volta sola, senza ack. Può perdersi.	Telemetria continua dove perdere un campione non è grave
1	At least once	Il broker invia ack. Se non arriva, il client ritrasmette. Il messaggio può arrivare duplicato.	Stato di un dispositivo, allarmi
2	Exactly once	Handshake a 4 vie. Garanzia di consegna unica. Overhead maggiore.	Comandi critici (pagamenti, attuatori non idempotenti)

Nel progetto:

- **Pubblicazione dei sensori:** implicito QoS 0 (basta la prossima lettura per “recuperare”).
- **Sottoscrizioni di termostato e monitor:** QoS 1, perché perdere un’anomalia è grave e un duplicato è innocuo.
- **Messaggi di status:** QoS 1 e flag `retained` (vedi sotto).

2.5 Retained message

Quando si pubblica con `retained = true`, il broker **conserva l’ultimo messaggio** su quel topic. Ogni nuovo subscriber che si iscrive lo riceve **immediatamente**, senza dover aspettare la prossima pubblicazione.

È perfetto per i messaggi di stato: se il monitor parte ora e vuole sapere se la temperatura è “online”, non deve aspettare il prossimo keep-alive — il broker glielo dà subito.

2.6 Last Will & Testament (LWT)

Cosa succede se un ESP32 perde corrente o cade dalla rete senza disconnettersi pulitamente? Il broker se ne accorge solo dopo il timeout del keep-alive.

Per gestire la cosa, MQTT permette al client, **al momento della connessione**, di lasciare un “testamento”: un messaggio (topic + payload + qos + retain) che il **broker pubblicherà al posto suo** se rileva la disconnessione anomala.

Nel progetto ogni nodo registra come LWT:

```
topic: casa/<nodo>/status
payload: "offline"
retain: true
qos: 1
```

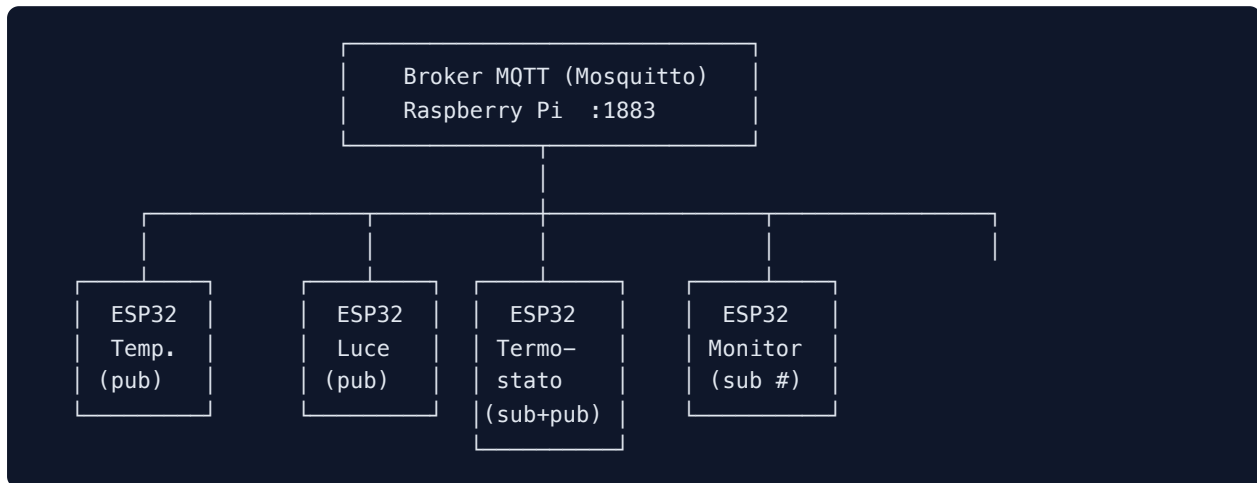
Appena la connessione va a buon fine, il nodo pubblica subito **"online"** (sovrascrivendo l'eventuale **"offline"** precedente). Risultato: leggendo **casa/+/status** si ha sempre lo stato attuale di ogni nodo della casa.

2.7 Keep-alive

A intervalli regolari (qui 30 s), il client invia un piccolo pacchetto **PINGREQ** al broker. Se per $1.5 \times \text{keep-alive}$ non riceve risposta, considera la connessione caduta. Serve a rilevare situazioni in cui il TCP non si è ancora accorto della disconnessione (es. cavo staccato senza FIN).

3. Architettura del sistema

3.1 Schema generale



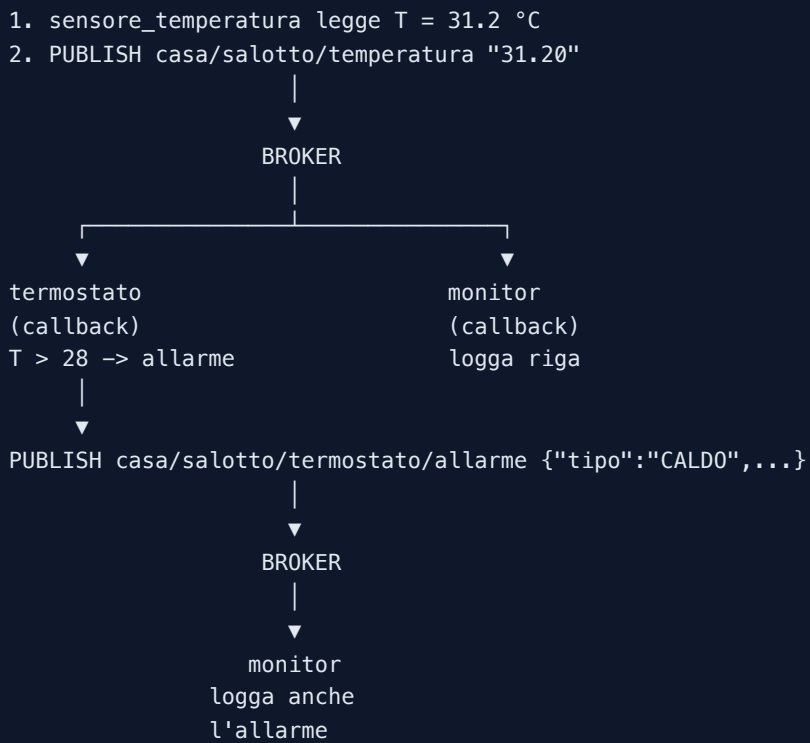
3.2 Mappa dei topic

Topic	Direzione	Payload	QoS	Retained
<code>casa/salotto/temperatura</code>	sensore → broker	<code>float</code> °C (es. <code>23.45</code>)	0	no
<code>casa/salotto/luce</code>	sensore → broker	<code>float</code> lux (es. <code>512.0</code>)	0	no
<code>casa/salotto/termostato/allarme</code>	termostato → broker	JSON	0	no
<code>casa/<nodo>/status</code>	nodo → broker (via LWT)	<code>online</code> / <code>offline</code>	1	sì
<code>casa/#</code>	broker → monitor	qualunque dei precedenti	1	—

Il JSON di allarme ha questa forma:

```
{"tipo":"CALDO","valore":31.20,"min":18.0,"max":28.0}
```

3.3 Flusso tipico di un evento “anomalia di temperatura”



4. Implementazione dei nodi

Ogni nodo è uno **sketch Arduino indipendente** (cartella separata, vincolo dell'IDE). Tutti condividono lo stesso scheletro:

1. configurazione (WiFi, broker, topic);
2. funzione di connessione WiFi con timeout;
3. funzione di connessione MQTT con LWT;
4. logica specifica del nodo;
5. `setup()` di inizializzazione;
6. `loop()` non bloccante che mantiene le connessioni vive.

Analizziamo i pezzi più significativi.

4.1 Connessione WiFi resiliente

Tutti i nodi usano la stessa funzione:

```
void connectWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 20000) {
    delay(500);
    Serial.print(".");
  }
  // se non si è connesso, non blocca: il loop() ritenterà
}
```

Punti chiave:

- `WIFI_STA` significa “station”, cioè client (l'alternativa sarebbe creare un access point);
- il `while` ha un **timeout di 20 secondi**: se la rete non risponde, la funzione esce senza freeze il dispositivo;
- nessun `while(true)` infinito: chi chiama (`loop()`) ritenterà al prossimo giro.

4.2 Connessione MQTT con Last Will

```
void connectMQTT() {
    while (!mqtt.connected()) {
        bool ok = mqtt.connect(
            MQTT_CLIENT_ID,
            nullptr, nullptr,           // niente user/password
            TOPIC_STATUS,              // will topic
            QOS_LEVEL,                 // will qos
            true,                       // will retain
            "offline"                   // will message
        );

        if (ok) {
            mqtt.publish(TOPIC_STATUS, "online", true);
        } else {
            Serial.printf("FAIL rc=%d\n", mqtt.state());
            delay(3000);
            if (WiFi.status() != WL_CONNECTED) return; // esce se il WiFi è caduto
        }
    }
}
```

Punti chiave:

- la `connect()` di PubSubClient ha **8 parametri** (la versione completa che accetta il LWT);
- il return `false` significa connessione fallita; con `mqtt.state()` si ottiene un codice numerico (es. `-2` = TCP fallita, `5` = non autorizzato);
- backoff fisso di 3 s — semplice e sufficiente per LAN;
- **safety exit**: se nel frattempo è caduto anche il WiFi, si esce; sarà il `loop()` a gestirlo nel giusto ordine (prima WiFi, poi MQTT).

4.3 Loop non bloccante

```
void loop() {
    if (WiFi.status() != WL_CONNECTED) { connectWiFi(); return; }
    if (!mqtt.connected()) { connectMQTT(); }
    mqtt.loop();

    unsigned long now = millis();
    if (now - lastPublish >= PUBLISH_INTERVAL_MS) {
        lastPublish = now;
        publishTemperature(readTemperature());
    }
}
```

Punti chiave:

- niente `delay()` lunghi: `mqtt.loop()` viene chiamato spesso, così il client risponde tempestivamente ai PING del broker e processa i messaggi in arrivo;

- lo scheduling temporale si fa con `millis()` (overflow ogni ~49 giorni, gestito correttamente dalla sottrazione `now - lastPublish`).

4.4 Il publisher di temperatura

La funzione di lettura simulata è scritta per **stressare** il termostato:

```
float readTemperature() {
    static float base = 22.0f;
    base += (random(-100, 101) / 1000.0f);    // deriva ±0.1°C
    if (base < 18) base = 18;
    if (base > 28) base = 28;

    // 1 volta su 20: anomalia
    if (random(0, 20) == 0) return base + random(5, 12);
    return base;
}
```

In condizioni normali la temperatura oscilla nel range “sano” (18-28 °C). Ogni ~100 secondi viene generato un picco anomalo, garantendo che durante una dimostrazione si veda almeno un allarme.

La pubblicazione converte il `float` in stringa ASCII:

```
char payload[16];
snprintf(payload, sizeof(payload), "%.2f", t);
mqtt.publish(TOPIC_TEMPERATURA, payload, false);
```

Si usa `snprintf` con buffer fisso (non `String`) per:

- evitare frammentazione della heap (problema serio su microcontrollori);
- garantire il numero massimo di byte scritti (16 byte sono sufficienti per qualsiasi `float` formattato con 2 decimali, segno e terminatore).

4.5 Il subscriber: callback di PubSubClient

Quando un client è iscritto a un topic, la libreria chiama una **funzione di callback** registrata in `setup()`:

```
mqtt.setCallback(onMessage);
```

La firma di `onMessage` è fissata dalla libreria:

```
void onMessage(char* topic, byte* payload, unsigned int length);
```

Attenzione: il `payload` **NON** è terminato da `\0`. Trattarlo come stringa C senza copiarlo prima è un bug classico. La pratica corretta:

```
char buf[32] = {0};
unsigned int n = length < sizeof(buf) - 1 ? length : sizeof(buf) - 1;
memcpy(buf, payload, n);
// adesso buf è una stringa C valida
```

4.6 Termostato: validazione del payload

Prima di convertire il payload in `float`, controlliamo che sia effettivamente un numero:

```
char* end;
float t = strttof(buf, &end);
if (end == buf) { // strttof non ha trovato cifre
    Serial.println("[WARN] Payload non numerico");
    return;
}
```

`strttof` setta il puntatore `end` all'inizio del buffer se non ha letto neanche una cifra. Questo difende il termostato da:

- bug del publisher che invia “NaN”, “error”, ecc.;
- pacchetti corrotti;
- topic colpito da un client non autorizzato.

Una volta validato, il confronto con le soglie e la pubblicazione dell'allarme è diretto:

```
if (t < SOGLIA_MIN || t > SOGLIA_MAX) {
    const char* tipo = t < SOGLIA_MIN ? "FREDDO" : "CALDO";
    char alarm[96];
    snprintf(alarm, sizeof(alarm),
        "{\"tipo\":\"%s\", \"valore\":%.2f, \"min\":%.1f, \"max\":%.1f}",
        tipo, t, SOGLIA_MIN, SOGLIA_MAX);
    mqtt.publish(TOPIC_PUB_ALLARME, alarm, false);
    digitalWrite(LED_PIN, HIGH);
}
```

Il LED on-board offre un **feedback locale immediato** anche senza dashboard.

4.7 Monitor: wildcard e NTP

Il monitor si iscrive a `casa/#` (tutto). Per il timestamp usa **NTP** quando possibile:

```
configTime(GMT_OFFSET_SEC, DST_OFFSET_SEC, "pool.ntp.org");
```

`configTime` avvia in background la sincronizzazione e l'orologio interno dell'ESP32 viene aggiornato. La lettura è poi semplice:

```
struct tm tm;  
if (getLocalTime(&tm, 50)) { // attende max 50 ms  
    strftime(out, n, "%H:%M:%S", &tm);  
}
```

Se NTP non è ancora pronto (es. primissimi secondi dopo il boot) o non c'è internet, si usa un **fallback**: secondi dal boot (`millis()/1000`).

Inoltre il monitor amplia il buffer interno di PubSubClient:

```
mqtt.setBufferSize(512);
```

Default sono 256 byte: con topic lunghi più payload JSON si rischia di scartare messaggi.

5. Configurazione del broker Mosquitto

5.1 Installazione su Raspberry Pi

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
```

5.2 Configurazione di rete

Per permettere agli ESP32 della LAN di connettersi, in `/etc/mosquitto/conf.d/local.conf`:

```
listener 1883 0.0.0.0
protocol mqtt
allow_anonymous true
persistence true
persistence_location /var/lib/mosquitto/
log_dest file /var/log/mosquitto/mosquitto.log
log_type all
```

Spiegazione delle direttive:

- `listener 1883 0.0.0.0` — ascolta sulla porta 1883 di **tutte** le interfacce di rete (non solo localhost);
- `allow_anonymous true` — accetta client senza credenziali. Ok per lab didattico in LAN privata, NON in produzione;
- `persistence true` — i retained message vengono salvati su disco e sopravvivono al riavvio del broker.

Avvio e abilitazione al boot:

```
sudo systemctl enable --now mosquitto
sudo systemctl status mosquitto
```

Trovare l'IP del Raspberry (da mettere come `MQTT_BROKER` negli sketch):

```
hostname -I
```

5.3 Test dal terminale

In una shell, subscriber wildcard:

```
mosquitto_sub -h <IP_BROKER> -t "casa/#" -v
```

In un'altra, publisher manuale per forzare un allarme:

```
mosquitto_pub -h <IP_BROKER> -t "casa/salotto/temperatura" -m "35.0"
```

Si dovrebbe vedere sul Serial Monitor del termostato il messaggio ***** ALLARME CALDO** e sul subscriber del terminale il JSON dell'allarme.

6. Gestione degli errori – riepilogo

Riepilogo delle tecniche difensive applicate nel progetto e del problema che risolvono:

Tecnica	Dove	Problema risolto
Riconnessione WiFi con timeout	tutti i nodi	AP momentaneamente irraggiungibile non blocca il device
Riconnessione MQTT con backoff 3 s	tutti i nodi	Broker spento o sovraccarico: si ritenta senza saturare la rete
Safety-exit se WiFi cade dentro <code>connectMQTT()</code>	tutti i nodi	Evita loop infinito di tentativi MQTT su WiFi morto
Last Will & Testament	tutti i nodi	Disconnessione hardware (corrente staccata) viene comunque notificata
Retained “online/offline”	tutti i nodi	Stato dei nodi disponibile subito a qualsiasi nuovo subscriber
<code>keepAlive(30)</code> + <code>socketTimeout(10)</code>	tutti i nodi	Rilevamento connessioni “fantasma” (TCP morto senza FIN)
QoS 1 sulle subscribe	termostato, monitor	Niente eventi critici persi
Validazione payload con <code>strtof</code>	termostato	Payload corrotti o non numerici non causano crash
<code>snprintf</code> con buffer fisso	tutti i nodi	Niente frammentazione heap, niente buffer overflow
<code>mqtt.setBufferSize(512)</code>	monitor	Topic + JSON lunghi non vengono scartati
Re-subscribe dopo riconnessione	termostato, monitor	Sessione non persistente: dopo reconnect le sub vanno rifatte
Contatore <code>messageCount</code>	monitor	Individua perdite di messaggi durante il debug
Scheduling con <code>millis()</code>	publisher	Niente <code>delay()</code> lunghi che bloccherebbero <code>mqtt.loop()</code>

7. Test e validazione

7.1 Avvio del sistema

L'ordine consigliato di accensione:

1. Raspberry Pi con Mosquitto;
2. Monitor (così cattura anche i primi eventi);
3. Termostato;
4. Sensori.

A regime, sulla seriale del monitor si vede qualcosa come:

```
[10:42:13] #001 casa/salotto/temperatura/status      -> online
[10:42:13] #002 casa/salotto/luce/status            -> online
[10:42:13] #003 casa/salotto/termostato/status      -> online
[10:42:13] #004 casa/monitor/status                 -> online
[10:42:18] #005 casa/salotto/temperatura            -> 22.34
[10:42:18] #006 casa/salotto/luce                   -> 487.0
[10:42:23] #007 casa/salotto/temperatura            -> 22.41
[10:42:23] #008 casa/salotto/luce                   -> 502.0
...
[10:43:08] #019 casa/salotto/temperatura            -> 31.07
[10:43:08] #020 casa/salotto/termostato/allarme     ->
{"tipo":"CALD0","valore":31.07,"min":18.0,"max":28.0}
```

7.2 Casi di test eseguiti

#	Scenario	Risultato atteso	Esito
1	Avvio normale	Tutti i nodi pubblicano "online", il monitor li registra	OK
2	Picco anomalo casuale del sensore	Allarme CALDO pubblicato e loggato, LED termostato acceso	OK
3	Spegnimento manuale del termostato	Dopo ~45 s il broker pubblica "offline" via LWT	OK
4	Riaccensione del termostato	Status torna "online", subscribe ricomincia	OK
5	Mosquitto fermato e riavviato	Sensori ritentano ogni 3 s; al ripristino tutto torna live	OK
6	Publish manuale di payload non valido ("ABC")	Termostato logga [WARN] e non crasha	OK
7	Disconnessione fisica del WiFi dell'ESP32	Ritenta in loop, non si blocca; al ripristino torna normale	OK

8. Conclusioni e possibili estensioni

Il progetto realizza un sistema IoT completo, distribuito e robusto utilizzando solo strumenti gratuiti e standard di settore. Il pattern publish/subscribe si è dimostrato ideale per il contesto: aggiungere un nuovo nodo (un altro sensore, una dashboard, un attuatore) non richiede modifiche ai nodi esistenti, basta scegliere un topic appropriato.

Possibili estensioni

- **Hardware reale:** sostituire le funzioni `readTemperature()` / `readLight()` con letture da sensori veri (DHT22, BH1750).
- **Sicurezza:** passare a **MQTT su TLS** (porta 8883) e abilitare l'autenticazione con username/password o certificati client.
- **Persistenza:** collegare al broker un servizio (Node-RED, Home Assistant, Telegraf+InfluxDB) per memorizzare lo storico e creare dashboard con grafici.
- **Attuatori:** un nodo aggiuntivo iscritto a `casa/salotto/termostato/allarme` potrebbe pilotare un relè per attivare un ventilatore o un riscaldamento.
- **Notifiche push:** un piccolo script in Python con `paho-mqtt` può rilanciare gli allarmi via Telegram, email, ecc.

Riepilogo dei concetti chiave appresi

- Architettura **publish/subscribe** vs request/response.
- Topic gerarchici e **wildcard** (`+` , `#`).
- Trade-off di **QoS** 0/1/2.
- **Retained messages** e **Last Will & Testament**.
- Programmazione Arduino non bloccante con `millis()` .
- Resilienza di rete su microcontrollori (WiFi + MQTT con riconnessione).
- Pratiche difensive in C/C++ embedded: buffer fissi, `snprintf` , validazione input.